SPETC v10 — Maintainer's Guide

Mauro Barbieri
mauro.barbieri@pm.me

2007–2026

Abstract

Internal documentation for whoever maintains or extends SPETC: the architecture and data flow, a module-by-module reference of every source file and its public functions, the on-disk data and configuration formats, the test suites, recipes for the most common modifications, and how to run and package the program on Windows and macOS.

Contents

1 Architecture

1.1 Layering

SPETC is a flat Python package (no sub-packages) with a strict one-directional dependency rule: **physics modules never import the GUI**, and only `etc_gui.py` imports Tk. The layers, bottom-up:

1. **Foundations**: `spectral_utils.py` (unit-aware curves, coverage checks, file readers) and `observing_conditions.py` (scintillation, refraction/dispersion, quantization). No project dependencies except each other's siblings.
2. **Physics core**: `etc_physics.py` (radiometry, PSF, S/N), `sky_brightness.py` + `sky_background.py` (sky model), `ephemeris.py` (astropy coordinates, refraction, airmass, parallactic angle), `atmospheric extinction` lives in `etc_physics.atmospheric_transmission` with curve loading in `spectral_utils`.
3. **Engines**: `photometry.py` (`PhotometryETC`) and `spectroscopy.py` (`SpectroscopyETC`); `detector.py` (`Detector`); `solvers.py` (closed-form exposure-for-S/N).
4. **Catalogues**: `star_catalog.py` (template database), `filter_catalog.py` (SVO VOTable profiles), `config_manager.py` (built-in observatory presets).
5. **Presentation**: `etc_gui.py` (Tk application, config persistence, orchestration) and `matplotlib_plots.py` (native plot windows).

1.2 Data flow of one calculation

1. `etc_gui._run_etc` validates inputs, resolves the target track with `ephemeris.compute_target_track` (refraction, Pickering airmass, parallactic angle per sample).
2. `_sky_models_for_track` converts the ICRS target to ecliptic/galactic latitudes and builds, per time sample, an observed-sky dictionary (brightness, colour model, spectral F_λ) from `sky_background` + `sky_brightness`.
3. Per time sample, an engine object is built with four plain dictionaries — `telescope`, `detector` (a `Detector`), `atmosphere`, `sky_model` — and returns a result dict (photometry) or `pandas.DataFrame` (spectroscopy).
4. The GUI assembles the time-series table, feeds the plot windows and the Results window, and persists the configuration.

The four-dictionary interface is the API contract: anything scriptable headlessly (tests, batch use) constructs those dictionaries directly, as `self_check.py` does.

1.3 Conventions

Wavelengths are Ångström internally; every user-supplied curve declares its unit explicitly and is converted once on load. Radiometric integrals carry `astropy.units`. Missing wavelength coverage is a hard `ValueError`, never a silent zero. Azimuth N=0/E=90; times are UTC inside, converted for display only.

2 Module reference

2.1 spectral_utils.py

Unit-aware spectral curves. `SpectralCurve` (frozen dataclass, strictly increasing Å grid); `make_curve`/`as_curve` validate and convert; `require_coverage` and `interpolate_checked` enforce the no-extrapolation rule; `interpolate_zero_filled` exists only for diagnostic plots; `load_two_column_curve` reads whitespace/comma text files; `load_fits_transmission_curve` reads atmospheric FITS tables (direct transmission columns, Paranal `EXTINCTION` mag/airmass converted once, or a $2 \times N$ image with `CUNIT1`).

2.2 observing_conditions.py

Pure functions, no state: `scintillation_fractional_rms` / `scintillation_variance_e2` (Young 1967 law), `scintillation_variance_rate_e2_s` (the same variance expressed as a time-linear rate, $(\dot{S}C)^2/2$, so closed-form solvers can absorb it as an extra Poisson-like term), `digitization_noise_e` ($g/\sqrt{12}$), `refractive_index_minus_one` and `differential_refraction_arcsec` (Filippenko 1982).

2.3 etc_physics.py

`magnitude_f_lambda` (zero-point F_λ), `synthetic_magnitude` (SVO photon/energy convention), `calibrated_template_magnitude` (CALSPEC m_{v0} + synthetic colour scaling), `collecting_area`, `atmospheric_transmission` (T^X), `instrument_transmission`, `electron_rate` (Eq. 4 of the physics document), `psf_encircled_energy` and `psf_slit_throughput` (Gaussian/Moffat; the Moffat slit coupling is the Student-t CDF with $\nu = 2\beta - 2$), and `snr` (CCD equation with an `extra_variance_e2` hook). The Gaussian-only wrappers `gaussian_encircled_energy` and `slit_throughput` are kept for backward compatibility.

2.4 sky_brightness.py / sky_background.py

`sky_brightness.py` owns the shared constants (solar minima, U–K tables, lunar factors) and the component physics: `van_rhijn_factor`, `zodiacal_s10`, `starlight_s10`, `dark_sky_position_correctio` `moonlight_scattering_function` and the nine-band `sky_brightness_total`. `sky_background.py` supplies the observed normalization `sky_magnitude_vega`: dark table + position/van-Rhijn correction below $-18°$ Sun altitude, Weaver daylight above the horizon, linear flux blend in twilight. The GUI stitches them: the ING normalization sets the level, the Benn–Ellison/K&S bands set the colour and the Moon term.

2.5 ephemeris.py

`compute_target_track` is the workhorse: refraction-corrected `AltAz` frames (`site_pressure_hpa`/`site_te` ISA at elevation), `pickering_airmass` above the $5°$ cutoff, Sun/Moon positions, Moon phase and separation, and `parallactic_angle_deg` per sample. Parsers and converters (`parse_ra`, sexagesimal formatters, frame converters) and the `find_*` rise/set helpers complete the module.

2.6 photometry.py / spectroscopy.py

Each exposes one class with one compute method taking the four-dictionary context plus per-call arguments. `PhotometryETC.compute_photometry_single` handles point and extended geometry, the full noise budget and saturation; `SpectroscopyETC.compute_spectroscopy` handles slit (PSF + dispersion-offset losses) and slitless (grating physics via module function `grating_dispersion_aa_per_pixel`) modes and returns a DataFrame whose `attrs` carry per-run metadata (mode, R, sky brightness, noise terms, dispersion).

2.7 detector.py, solvers.py

`Detector` stores pixel size, gain, full well, bit depth, read noise, dark current; `counts_to_adu` and `saturation_flag` (`FULL_WELL`/`ADC`/`BOTH`/`NONE`); `load_gain_table` reads the per-setting CMOS tables (Sect. 3). `solvers.exposure_time_for_snr` inverts the CCD equation in closed form, with `extra_variance_rate_e2_s` absorbing time-linear non-Poisson terms (scintillation); `solvers.plan_stack` produces the sub-exposure/frame-count plan with the read-noise penalty and the sky-limited frame length, on the same closed form.

2.8 make_filter_profile.py, spetc_batch.py

Standalone tools sharing the physics modules. `make_filter_profile` turns a two-column transmission file into a full SVO-format VOTable pair: `measure_band_quantities` evaluates every SVO characterising quantity (Min/Max/Peak on the tabulated grid, per SVO's literal definitions) and `synthetic_vega_zero_point_jy` computes the Vega zero point from the bundled CALSPEC spectrum; validated against the shipped 2MASS.H profile in the test suite. `spetc_batch.run_batch` drives both engines headless from a JSON description and renders the result CSV plus a self-contained HTML summary with an embedded S/N figure — it is also the reference example of the four-dictionary engine API.

2.9 star_catalog.py, filter_catalog.py, config_manager.py

Catalogue parsing (`interpola.db.csv` → `StarRecord`; spectrum loading with monotonicity cleanup), SVO VOTable parsing (`FilterProfile` with zero point, `DetectorType`, metadata in Å; `BLANK` special-cased to the AB scale), and the built-in observatory presets from `observatories.json`.

2.10 etc_gui.py, matplotlib_plots.py

The Tk application: `_var(key, default)` registers every setting for automatic JSON persistence; `_run_etc` orchestrates; the helper methods `_atmosphere_dict`, `_source_geometry_kwargs`, `_sky_models_for_track`, `_photometry_time_series`, `_spectroscopy_time_series` and `_build_time_series_dataframe` are the seams to modify first. `matplotlib_plots.ETCPlotWindow` renders the metric/altitude/sky-path (and spectrum) panels with the manual time slider.

3 On-disk formats

3.1 data/ directory

interpola.db.csv Header line then comma-separated rows: `nrow, Vmag, B-V, name, spt, filename`; `filename` is relative to `data/` (subdirectories allowed, e.g. `bpgs/bpgs_92.ascii`). `Vmag` is the CALSPEC visual magnitude represented by the file (m_{v0}).

template spectra Two-column ASCII: wavelength [Å], calibrated F_λ. Non-finite and non-positive rows are dropped; wavelengths are sorted and deduplicated on load.

qe.dat Two-column wavelength/QE; the wavelength unit is declared in the configuration (`Angstrom`/`nm`/`um`).

filters.list One entry per line naming SVO profiles; entries with or without the `filters/` prefix and `.xml` suffix all resolve. Each logical filter needs `<name>_Vega` and `<name>_AB` VOTables carrying `MagSys`, `ZeroPoint` [Jy], `DetectorType`, `WavelengthUnit` and the transmission table. `BLANK.xml` (unity 3000–26000 Å, AB) is optional and auto-detected.

earth_atmospheric_transmission.fits Optional; accepted layouts in Sect. 2 (`spectral_utils`).

CMOS gain tables One row per gain setting, whitespace- or comma-separated, # comments allowed: `setting e-/ADU read_noise_e full_well_e`. Loaded through `detector.load_gain_table`; the GUI persists the path in `gain_table_path` and restores it at start-up.

Batch run descriptions JSON, see `examples/`: top-level `mode`, `telescope`, `detector`, `atmosphere`, `sky` (fixed_ab/sqm), `target`, `photometry` or `spectroscopy` blocks, optional `target_snr` and `stack_sub_exposure_s`.

3.2 Configuration and profiles

`etc_user_config.json` is a flat string-to-string map of every registered GUI variable plus `schema_version`; it is rewritten after each successful run and on exit. Instrument profiles are the same keys restricted to the instrument subset (`_instrument_profile_keys`), saved with relative copies of the QE, response and slit-R curves beside them. `etc_sites.json` holds user sites (`name/lat/lon/elev/utc_offset_h/timezone`) `observatories.json` the built-in presets. When adding a new persistent setting, register it with `self._var("key", default)` — persistence is automatic; bump `schema_version` only for incompatible changes.

4 Modification recipes

4.1 Add a filter

Download the `_Vega` and `_AB` VOTables from the SVO Filter Profile Service, drop them into `data/filters/` and append the two names to `data/filters.list`. Nothing else: the selector list is rebuilt at start-up and all metadata (zero point, pivot, detector type) comes from the files.

4.2 Add a template spectrum

Place the two-column ASCII under `data/` and append a row to `interpola.db.csv` with its row count, the visual magnitude the file is calibrated to (m_{v0}; 0 for a visual-zero distribution), $B - V$, name, spectral type and relative path.

4.3 Add a site

Prefer the GUI (Save current site); or edit `etc_sites.json`. `observatories.json` is for shipped defaults only.

4.4 Add a noise term

Implement the variance in `observing_conditions.py` as a pure function returning e^{-2}; add it to the `extra_variance_e2` sums in `photometry.py` and `spectroscopy.py`; expose any new inputs through `_atmosphere_dict` in the GUI; extend `tests/test_spectral_pipeline.py` with an order-of-magnitude check; document it in the physics document's Table 3.

4.5 Add a PSF model

Extend `psf_encircled_energy` and `psf_slit_throughput` in `etc_physics.py` with the new `psf_model` branch (the slit coupling needs the 1-D marginal CDF, analytic or numeric), add the name to the GUI combobox, and add a regression test comparing it against the Gaussian in a limit.

4.6 Change the sky model

All component physics lives in `sky_brightness.py`; the normalization/blending policy in `sky_background.py`; the GUI merge in `_sky_models_for_track`. A future spectral sky model should populate the existing `spectral_sky_f_lambda` key of the sky-model dictionary — the spectroscopy engine already consumes it.

4.7 Coding rules

Match the existing style: physics as pure functions with explicit units, engines free of Tk, no silent fallbacks (raise `ValueError` with the offending numbers), constants named after their paper, and every new physical relation cited to its reference in a docstring and covered by a test pinned to a published value.

5 Testing

Two entry points, both dependency-light and network-free:

```
python3 self_check.py
python3 tests/test_spectral_pipeline.py
```

`self_check.py` is a synthetic end-to-end smoke test of both engines on the 358-mm reference configuration; it pins the scintillation-limited photometric S/N ($\sim 1.3 \times 10^3$) and the per-element spectroscopic S/N (hundreds). `tests/test_spectral_pipeline.py` holds the unit-level regressions: unit conversion of user curves, SVO photon/energy semantics, hard coverage failures, FITS atmosphere layouts, and the v10 physics checks — Krisciunas–Schaefer scattering values, ecliptic symmetry and position dependence of the sky, van Rhijn bounds, Pickering vs. sec z, parallactic-angle sign convention, Moffat wings and decentred coupling, Filippenko dispersion scale, scintillation and quantization magnitudes, SA100/SA200 dispersion, and extended-source area fractions. The file inserts the repository root on `sys.path`, so it runs from any working directory; it is also `pytest`-compatible (every test is a plain `test_*` function).

Rules: any bug fix gains a regression test that fails on the pre-fix code; any new physical relation gains a test pinned to a published number, not to the code's own output. The GUI is deliberately untested — keep logic out of it and in the engines, where it is testable headlessly.

6 Running and packaging on Windows and macOS

The code is pure Python (Tk, NumPy/SciPy/pandas, Astropy, Matplotlib, astroquery) with no compiled components of its own and no platform conditionals; porting is an installation exercise.

6.1 Windows

1. Install CPython 3.10+ from python.org (the installer bundles Tk; tick "Add python.exe to PATH"). Avoid the Microsoft Store build, whose sandbox complicates file dialogs and per-user paths.
2. `py -m pip install -r requirements.txt`
3. Run with `py etc_gui.py` from the distribution folder.

Notes: paths are handled with `pathlib`, so backslashes are transparent; configuration files are written next to the program — install under a user-writable directory, not `Program Files`; on high-DPI displays Tk scales automatically on Python $\geq$ 3.12, otherwise call `ctypes.windll.shcore.SetProcessDpiAwareness(` early if the UI looks blurry. For a self-contained distribution use PyInstaller:

```
pip install pyinstaller
pyinstaller --windowed --name SPETC ^
    --add-data "data;data" ^
    --add-data "etc_sites.json;." ^
    --add-data "observatories.json;." etc_gui.py
```

Matplotlib and Astropy need no hooks beyond PyInstaller's built-ins; test the frozen build offline, because Astropy must fall back to its bundled IERS table (the code already sets `auto_download = False`).

6.2 macOS

1. Install CPython from python.org (universal2; bundles a private Tcl/Tk 8.6 — do *not* rely on the Apple system Python). Homebrew works too: `brew install python-tk`.
2. `python3 -m pip install -r requirements.txt`
3. `python3 etc_gui.py`

Notes: on Retina displays Tk renders sharp with the python.org build; Gatekeeper will quarantine a PyInstaller app (`--windowed` produces `SPETC.app`) unless it is signed and notarized — for personal use, right-click → Open once, or clear the quarantine attribute with `xattr -dr com.apple.quarantine SPETC.app`. The case-sensitive file systems sometimes used on macOS are already handled: all data files are referenced with their exact case.

6.3 Both platforms

Keep `data/` beside the executable (the shipped data-directory entry is resolved relative to `etc_gui.py`); time zones use the standard `zoneinfo` database (on Windows, `pip install tzdata` is pulled in automatically as a dependency of the requirements); and verify a port with the two test programs before first use — they exercise the whole physics stack without opening a window.